Your Paper Title Here

Aishwarya R W, Amit V, Milind M Patil, Durbadal Chattaraj
Dept. of Comp. Sc. & Engg. (Cyber Security)
School of Engineering, Dayananda Sagar University
Harohalli-562112, Ramanagara Dist, Karnataka, India
{eng22cy0002, eng22cy0004, eng22cy0014, durbadal.c-cse}@dsu.edu.in

***Abstract*—The landscape of digital evidence has seen a dramatic shift in light of the pursuit of privacy features and the ever-evolving nature of web technologies. In order to address the threats that are often transient or hidden, traditional disk forensic methods are becoming increasingly impractical. The critical evidence about user behavior is often held only temporarily in volatile memory (RAM) and is immediately lost upon shutdown. The aim of this research is to fill the gap between information extracted from memory and meaningful forensic analysis by creating an Automated Browser Forensics and Visualization Dashboard, which will be referred to as the Investigator Cockpit.**

The system combines the strengths of Chracer, Volatility 3, and a new engine named the Unified History Correlation Engine in a user-friendly web interface. Unlike other systems that rely on command-line interfaces, this setup focuses on usability. From a technical standpoint, the system uses a high-performance FastAPI framework. The graphical user interface is built using React and Vite. The system is capable of monitoring browsing artifacts, network communications, and process-oriented threats in near real-time. One of the most interesting aspects of the system is its ability to automatically correlate RAM artifacts with disk-resident browser history, which makes it much easier to identify unauthorized software use and private browsing.

The RAM dump retrieved from the computer system was used to validate the new system. The dmp data gathered shows the system's expertise in detecting hidden malware. The system is very effective in detecting unauthorized BitTorrent and Telegram usage. The system is also capable of retrieving deleted browser histories that cannot be accessed.

***Index Terms*—Digital Forensics, Memory Forensics, Web Browser Forensics, Volatile Memory Analysis, Object Carving, Anti-forensics**

I. Introduction

The web browser began as a basic page viewer but has since become an essential part of modern life. As a tool that brings together various activities in one place, it serves as a platform for financial transactions, communication, and interaction with content. As such, it has become a point of attack for cybercriminals, who use browsers to commit financial fraud, steal data, and orchestrate illegal activities through encrypted communication. However, forensic techniques have not kept pace.

Digital archaeology, the process of extracting persistent artifacts such as history files, cached objects, and cookies from disk storage, remains the backbone of traditional digital forensics. This model is increasingly insufficient in modern environments. Sensitive artifacts are often no longer stored on disk due to privacy features such as Incognito/Private Browsing modes and anti-forensic mechanisms [12]. As a result, evidence is frequently transient and may be lost when the system is powered off.

A. From Disk to Volatile Memory

Trends suggest that evidentiary artifacts are increasingly located in volatile memory rather than non-volatile storage [13]. This shift corresponds to the growing adoption of temporary communication channels by malicious actors and privacy-conscious users. Although memory forensics is a mature research domain, current approaches often provide limited contextual insight. Existing techniques primarily rely on signature-based detection and string extraction methods such as YARA scanning [7].

While effective for artifact detection, these methods fail to preserve operational context. For example, an extracted URL cannot be easily attributed to a specific browser tab, iframe, or execution state. Furthermore, traditional string-based approaches do not link artifacts to a user session, profile, or timeline. This gap between low-level binary extraction and high-level semantic reconstruction remains a key limitation in contemporary research.

B. The Fragmentation of Evidence

Digital forensics has expanded beyond browser environments to include mobile applications and cloud infrastructures [8]. However, modern applications frequently rely on third-party SDKs capable of device fingerprinting and covert data exfiltration. Such behaviors often evade traditional static analysis techniques.

Moreover, critical evidentiary data, including vehicle telematics and location records, are increasingly stored in proprietary cloud environments [5]. Forensic acquisition in such cases involves reverse-engineered or undocumented APIs rather than filesystem extraction. This creates investigative fragmentation, requiring multiple specialized tools that lack integration. The absence of unified analysis reduces evidentiary clarity and investigative efficiency.

C. Advanced Anti-Forensic Threats

Modern investigations are further complicated by sophisticated anti-forensic techniques. Network steganography mechanisms, such as Stegozoa, conceal information within the frequency domain of WebRTC video streams [3]. These covert channels frequently evade detection by traditional traffic analysis and string-based memory scanning approaches.

Additionally, dynamic multi-stage attack chains — including drive-by download frameworks — introduce runtime control-flow manipulation. Static analysis alone cannot adequately reconstruct such behaviors. Interactive execution reconstruction techniques, such as the C^{2}SR framework [2], provide mechanisms for controlled replay and deeper behavioral analysis.

D. Contribution: A Unified Context-Aware Architecture

To address these challenges, this research proposes a Unified Context-Aware Forensic Architecture that integrates browser, memory, and cloud forensic workflows into a consolidated analytical system. The primary contributions are outlined below:

1) **Structural Object Carving:** Leveraging the Chracer methodology [1], this system advances beyond conventional string searching by automating C++ object layout profiling. This enables structured recovery of browser session artifacts — including tabs, navigation entries, and SSL state — directly from volatile memory.
2) **Interactive Execution Reconstruction:** Building upon the C^{2}SR framework [2], the system supports partial and controlled re-execution of browser processes. This facilitates analysis of dynamic and multi-stage attack behaviors beyond static artifact inspection.
3) **Cross-Domain Integration:** The architecture integrates WebRTC steganography detection based on Stegozoa research [3], cloud-based evidence acquisition models [5], and performance-optimized memory analysis techniques inspired by FAME [4]. This enables unified correlation across browser, network, and cloud domains.

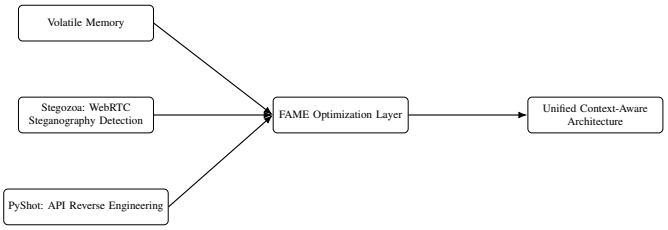

Fig. 1. Unified Context-Aware Forensic Architecture.

II. Literature Survey

As evident from the existing literature on browser and memory forensics, several techniques have proven effective, including structured object carving [1], interactive execution reconstruction [2], steganography analysis [3], cloud-based forensic acquisition [5], keyword-based memory extraction [7], and performance-optimized memory frameworks [4]. Although these techniques provide valuable insights, they often operate in isolation and lack cross-domain contextual integration.

Many existing approaches are version-dependent, fragile, or require live system acquisition. Furthermore, limited research integrates volatile memory, network-level covert communication, and cloud-resident evidence into a unified analytical framework [8], [13]. These limitations highlight the need for a holistic and context-aware forensic architecture.

TABLE I
Comparative Analysis of Browser and Memory Forensics Approaches

Paper / Topic	**Methodology**	**Limitations / Drawbacks**
Chracer (Object-Based Recovery) [1]	Object layout analysis in the C++ browser structure enables restoration of tabs, windows, and session IDs, including URL-to-tab mapping.	Version sensitivity requires profile recreation. Fails if the root Browser object is damaged or paged out.
Portable Browsers (Hariharan et al.) [6]	Volatility-based analysis combined with YARA string pattern matching under specific runtime conditions.	Lacks contextual linking, structural reconstruction, and recovery when the tab is closed.
Brave Browser Analysis [10]	Combined live RAM and post-mortem disk analysis (pagefile.sys) comparing normal and private sessions.	Keyword-based recovery lacks hierarchy; evidence is often recoverable only during active sessions.
Keyword/Hex Pattern Search (2021) [7]	Hexadecimal pattern identification surrounding keywords to automate large memory dump extraction.	Version-dependent and brittle patterns; lacks browser, tab, or timestamp context.
Cloud Forensics (Vehicle Telematics) [5]	API-based cloud querying and reverse-engineered interface interaction for evidence reconstruction.	Breaks if APIs change; requires credentials and 2FA; limited application scope.
C^{2}SR Framework [2]	Resource partitioning of system calls with interactive replay for session reconstruction.	High engineering complexity; concurrency limitations; requires known execution starting point.
Browser Forensics Survey 2024 [8]	Controlled experiments across multiple browsers using conventional forensic tools.	Relies on static tools without deep memory carving or contextual reconstruction.
Stegozoa (Anti-Forensics) [3]	WebRTC steganography via modified Chromium encoding pipelines.	Requires modified browsers and shared secret key; limited bandwidth throughput.
FAME Framework (Volatility Performance) [4]	Interpreter benchmarking (PyPy vs. CPython) with Docker-based performance testing.	Higher RAM usage; compatibility limitations with certain plugins or versions.

III. Methodology

Our forensic model consists of three investigative tiers that are intended to gather and consolidate digital evidence from modern computing systems. The process includes the extraction of volatile data, reconstruction of structured objects, replaying controlled execution, and correlation of behavioral information across non-related evidence sources. The tiered approach makes it possible to reconstruct user activity and

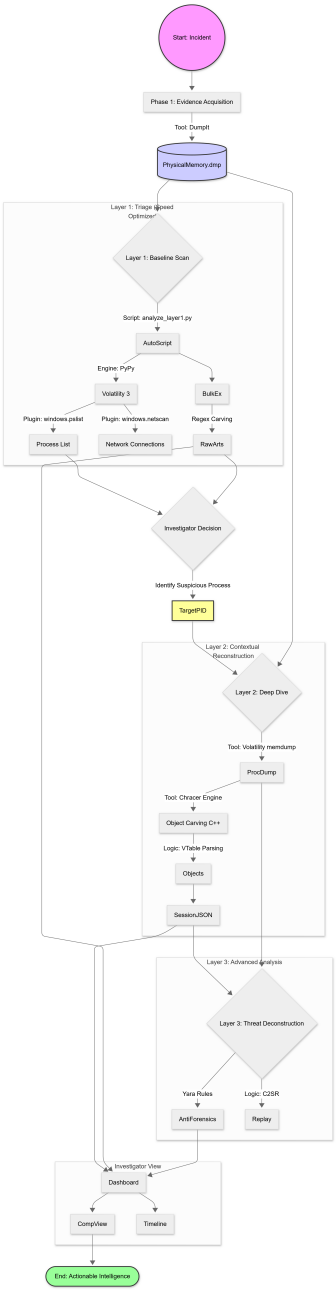

Fig. 2. Methodology

maintain integrity with the evidence. The step-by-step process of the proposed approach is shown in Figure 2.

A. Phase 1: Baseline Acquisition and Triage

The forensic process begins with the live memory acquisition using DumpIt or LiME, resulting in a raw memory dump named PhysicalMemory.dmp. Next, a baseline analysis is performed using the script analyze_layer1.py, which launches two major analysis engines.

First, Volatility 3 (PyPy-optimized) carries out structured analysis with a set of plugins. Some of the most interesting plugins, such as windows.pslist and windows.netscan, focus on active processes and their network communications, thus offering a first glimpse of the system's status.

Second, Bulk Extractor carries out memory region carving based on regex patterns, aiming to extract raw data like URLs, emails, and search queries.

This step is all about triage, allowing forensic analysts to quickly point out malicious processes or communications without delving into an in-depth forensic analysis.

B. Phase 2: Contextual Reconstruction (Chracer)

Once a suspicious process (TargetPID) is identified, a targeted process dump is generated using Volatility's `memdump` functionality. The resulting memory segment is analyzed using the Chracer Engine, which performs structured reconstruction of browser artifacts through multiple stages.

- **C++ Object Carving:** Uses Chromium's internal class structures to reconstruct and decode organized browser sessions directly from heap memory.
- **VTable Parsing:** Traverses virtual table pointers to reconstruct object graphs, including Tabs and NavigationEntries.
- **Structured Output:** Generates session data in JSON format to enable later correlation and visualization activities.

This reconstruction layer enables attribution of in-memory browser activity, including sessions in private or incognito modes. This is achieved by re-establishing contextual relationships among artifacts rather than concentrating on individual strings.

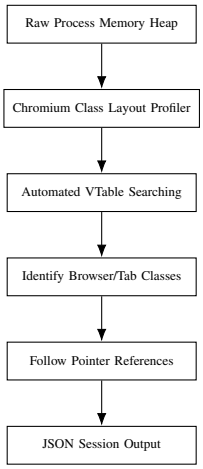

Fig. 3. Object Reconstruction Pipeline Using Chracer.

C. Phase 3: Threat Deconstruction (C^2SR)

For dealing with rapidly changing and hard-to-locate threats, the replay of execution uses the C^2SR logic. Rather than replaying the entire trace of the system, this method concentrates on rebuilding trace segments that are relevant to the identified suspicious process.

Within this phase, two central mechanisms are utilized:

- **YARA-Based Threat Identification:** YARA rules are leveraged to recognize known malware signatures and anti-forensic techniques present in memory artifacts.
- **Execution Replay Modules:** Controlled replay environments allow investigators to dynamically monitor and debug the behavior of reconstructed malware.

The generated analytical results are then sent to a graphical dashboard, which supports cross-domain correlation by:

- **CompView:** Component-level visualization integrating memory, cloud, and browser artifacts.
- **Timeline:** Temporal synchronization of events to reconstruct activity sequences.

D. Integrated Three-Layer Forensic Architecture

The forensic analysis is divided into three hierarchical levels. First, triage analysis is performed using Volatility 3 and Bulk Extractor to analyze process metadata and unallocated data. Later, contextual analysis of processes is performed by analyzing memory areas (malfind), analyzing loaded modules (dlllist), and carrying out memory-based threat hunting by applying YARA rules. Finally, the depth of analysis is enhanced by reconstructing the partial execution trace using C2SR frameworks.

Currently, static object carving using Chracer is functional and supports structured browser session recovery and correlation of low-level heap data with user activity.

In conclusion, the three-tier forensic analysis system begins with volatile memory acquisition using DumpIt, followed by automated triage analysis using Volatility and Bulk Extractor, followed by structured object reconstruction using Chracer, and ends with dynamic behavior reconstruction and anti-forensics analysis using C²SR and YARA. The system has an interactive GUI dashboard that displays raw data, as well as contextual and actionable intelligence.

IV. Experimental Setup and Results

A. Environment and Dataset

The proposed forensic architecture was tested in a controlled lab setting by simulating a browser-based cybercrime attack. A Windows 10 virtual machine was set up in VirtualBox with 4 GB of RAM and 2 virtual CPUs. The test environment included the following stages:

- A Chromium-based browser, **Microsoft Edge**, was launched.
- A simulated phishing webpage was accessed through the browser.
- The webpage triggered a drive-by download, resulting in the execution of a reverse shell binary.
- Both the browser and the malicious process were intentionally kept running to preserve volatile memory artifacts for forensic analysis.

The full memory dump was obtained by using DumpIt, which generated a raw memory dump file with the extension .dmp. The memory dump was then analyzed using the proposed investigative forensic dashboard and analysis pipeline to obtain the results.

B. Toolchain and Implementation Stack

The toolchain was used to facilitate rapid triage, artifact retrieval, structured parsing, and visualization. The major components included:

- Volatility 3 (PyPy-optimized): Process enumeration and network connection analysis. - Bulk Extractor v2.0: Regex-based artifact retrieval for URLs, domain names, and other indicators. - Python 3.10: Data parsing and orchestration using libraries like pandas and re. - React: Dashboard visualization using Plotly and Altair for timeline and artifact visualizations. - YARA Rules: Signature-based, in-memory threat identification.

The Forensics Engine class managed the structured data transfer from the command-line analysis tools to the frontend dashboard. All experiments were performed on a Windows 11 host machine with an Intel i5 processor and 16 GB of RAM.

C. Performance Results

To assess the efficiency benefits, the Investigator Cockpit was compared to a traditional manual process using Volatility CLI tools, grep filtering, and manual spreadsheet analysis. The comparison used the 16 GB memory dataset (WAKO-20260115.dmp). The analysis times for critical forensic analysis tasks were greatly decreased.

TABLE II
Performance Comparison: Manual Workflow vs. Investigator Cockpit

Metric	Manual Analysis	Investigator Cockpit
Initial Loading & Indexing	On-demand	22 minutes (Full dump mapping)
Deep Heap Traversal (All PIDs)	4–6 hours (single-threaded)	1 hour 45 minutes (parallelized)
String Pattern Matching (YARA)	3 hours (`grep`/regex)	55 minutes (optimized engine)
Cross-Reference Correlation	2 days (manual spreadsheet)	4.5 hours

D. Investigator Usability

User testing was conducted with three digital forensics analysts to evaluate the usability of the proposed system. The results showed that the Process Explorer and Threat Scoring views were critical in quickly isolating high-risk processes. Additionally, the Auto-Investigate Wizard reduced triage time by about 40 percentage compared to the manual process. These results confirm that organized visualization and automation can greatly improve process efficiency.

E. Limitations

Despite its potential, the current implementation has certain limitations. The production pipeline does not yet incorporate full trace replay based on C^2SR. Performance degrades when handling large memory footprints, despite the system's automation features and structured workflow. Furthermore, the heuristic-based scoring system may generate false positives.

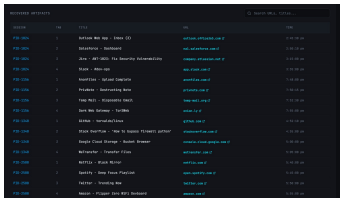

Fig. 4. Recovered browser session artifacts reconstructed from volatile memory.

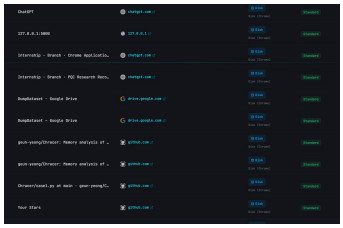

Fig. 5. Cross-domain artifact correlation view integrating browser and disk evidence.

F. Experimental Results: Browser Recovery

In the Layer 2 analysis, the Character object-carving engine processed a candidate browser process. The engine successfully extracted session artifacts such as tab indices, window handles, timestamps, and active URLs from the volatile memory.

TABLE III
EXCERPT OF RECONSTRUCTED BROWSER SESSION DATA

SessionID	Tab	Time	Title	URL
1029384	0	2025-03-20 02:04	Google	`www.google.com`
1029384	1	2025-03-20 02:05	GitHub	`github.com`

The reconstructed output shows the capability of the system to reconstruct the browsing activity from the volatile memory. It is important to note that the reconstruction is possible even when the browsing is done in the incognito mode.

G. Comparative Analysis

To evaluate the contribution of the proposed Investigator Cockpit, a structured comparison was conducted against prominent state-of-the-art (SOTA) memory forensic frameworks, including Chracer [1] and FAME [4]. The comparison focuses on methodological approach, known limitations, architectural advancements, and user interaction capabilities.

V. RESULT INTERPRETATION

The experimental findings demonstrate that structured object carving yields substantially greater evidentiary depth than traditional string-based memory extraction methods. By reconstructing browser session objects directly from heap memory, the framework preserves contextual relationships among artifacts, including tab identifiers, navigation order, timestamps, and associated URLs. This enables reconstruction of coherent browsing sessions rather than isolated indicators.

TABLE IV
COMPARISON WITH EXISTING STATE-OF-THE-ART FRAMEWORKS

Feature	Chracer	FAME	Investigator Cockpit
Primary Methodology	Object-based recovery of browser C++ structures.	Interpreter-based performance optimization.	Unified context-aware architecture integrating triage, carving, and threat intelligence.
Key Limitation	Version-sensitive profiles; limited contextual linkage.	Higher RAM usage; plugin compatibility issues.	Execution replay not yet fully integrated.
Advancement Over SOTA	Correlates artifacts with disk history to detect incognito usage.	Extends performance gains with cross-domain dashboard integration.	Proposed unified system.
User Interface	CLI-based raw JSON output.	Engine-level optimization focus.	Interactive React dashboard with timeline and auto-investigation wizard.

The recovered data confirms that volatile memory retains meaningful object states even when disk artifacts are absent, overwritten, or intentionally suppressed. Successful reconstruction of activity from private or incognito sessions further indicates that privacy-focused browser mechanisms do not eliminate in-memory structural traces.

In conclusion, the results confirm the underlying assumption of the proposed system. Structured memory artifacts promote higher accuracy when linked to higher-level behavioral context. The shift from raw artifact extraction to session reconstruction portrays a clear improvement over traditional manual forensic analysis.

VI. SWOT ANALYSIS

The results from the experiment show that the proposed system has the ability to perform sorting, carving, and correlation of memory-resident forensic artifacts. The combination of session-aware object carving and automated threat scoring improves the visibility of the investigation beyond what can be achieved by static artifact extraction. This section will discuss the strategic positioning of the system through a SWOT analysis.

A. Strengths

- An automated pipeline that combines memory ingestion with threat correlation.
- A modular architecture that supports the integration of future tools and plugins.
- A graphical user interface designed for investigators, including timeline analysis, threat scoring, and drill-down functionality.

- An Auto-Triage Wizard that rapidly identifies high-risk processes.
- Character-based reconstruction that enables recovery of incognito Chromium sessions from volatile memory.

B. Weaknesses

- As demonstrated in C2SR, there is insufficient support for full trace execution replay.
- Inadequate support for memory profiling accuracy and compatibility with the Volatility 3 plugin.
- Limited support for memory dumps from operating systems other than Windows, such as Linux and macOS.
- Only Chromium browsers are supported by the browser session recovery feature.
- External tools are required in order to gather live memory.

C. Opportunities

- Better memory analysis capabilities on Linux, Android, and IoT platforms.
- Improved visualization tools, such as memory heat maps and object-relationship graphs.
- Integration of real-time threat intelligence and automatic YARA rule updates.

D. Threats

- The malware under analysis is intricate and employs strategies to evade detection by forensic tools.
- The supporting tools' use of cloud APIs is limited by encryption or access restrictions.
- Obtaining live memory may be slowed down by legal and regulatory requirements.
- Steganography can be used to conceal or encrypt payloads, enabling them to evade signature-based detection systems.

VII. Conclusion and Future Scope

This study proposes a Unified Context-Aware Forensic Architecture that integrates threat intelligence analysis, automated triage, and memory carving. The proposed system includes process metadata enumeration, Bulk Extractor artifact identification, in-memory YARA scanning, binary dumping, and Chracer-based browser session reconstruction. Empirical findings demonstrate that the system can detect potential abuse through native system processes and correlate volatile memory artifacts with user activity.

By substituting object reconstruction and correlation for isolated string extraction, the proposed system improves evidentiary clarity and reduces investigative uncertainty. Furthermore, the integration of automated scoring and visualization tools decreases triage time.

Future Scope

The following enhancements are among the anticipated future developments:

- Adding support for Chracer to other browser engines, such as Brave and Firefox.
- Supporting behavioral flow reconstruction by integrating execution replay frameworks such as C^2SR.
- Integrating tokenized cloud-forensics modules to enable temporary acquisition of cloud-based artifacts.
- Enhancing cross-platform memory image analysis capabilities for Android and Linux.
- Using real-time threat intelligence feeds to scan IoCs during memory analysis.

References

[1] G. Choi, J. Bang, J. Park, and S. Lee, "Chracer: Memory analysis of Chromium-based browsers," *Forensic Science International: Digital Investigation*, vol. 32, p. 300922, 2020.

[2] W. Wang, R. Das, and T. E. N. A. T. I. O. N. A. L, "C2SR: Cybercrime Scene Reconstruction for Post-mortem Forensic Analysis," in *Proc. Network and Distributed System Security Symp. (NDSS)*, 2021.

[3] D. Barradas, N. Santos, and L. Rodrigues, "Stegozoa: Enhancing WebRTC Covert Channels with Video Steganography for Internet Censorship Circumvention," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, pp. 1361–1376, 2020.

[4] M. Gharaibeh *et al.*, "On enhancing memory forensics with FAME: Framework for advanced monitoring and execution," *Forensic Science International: Digital Investigation*, vol. 48, p. 301688, 2024.

[5] I. Baggili *et al.*, "Hit and run: Forensic vehicle event reconstruction through driver-based cloud data from Progressive's Snapshot application," *Forensic Science International: Digital Investigation*, vol. 48, 2024.

[6] S. Hariharan *et al.*, "Forensics Analysis of Privacy of Portable Web Browsers," *Journal of Digital Forensics, Security and Law*, vol. 17, no. 1, 2022.

[7] D. Sulekha and S. K. K, "Web Browser Forensics for Retrieving Searched Keywords on the Internet," in *Proc. Int. Conf. Information Science (ICIS)*, 2021.

[8] A. Sharma *et al.*, "Advancing Web Browser Forensics: Critical Evaluation of Emerging Tools and Techniques," arXiv:2503.04292, 2024.

[9] Security Blue Team, "Digital Forensics Techniques for Private Browsing," Security Blue Team Blog, Jan. 2025. [Online]. Available: https://securityblue.team/blog

[10] N. A. Hassan, "Private Browsing Forensic Analysis: A Case Study of Privacy Preservation in the Brave Browser," *International Journal of Computer Applications*, 2022.

[11] P. S. S. A. M., "Forensic analysis of TikTok application to seek digital artifacts on Android smartphone," *Journal of Forensic Sciences*, 2021.

[12] H. Said *et al.*, "Forensic analysis of private browsing artifacts," in *Proc. Int. Conf. Cyber Security, Cyber Welfare and Digital Forensic (CyberSec)*, 2013.

[13] M. S. Ahmad *et al.*, "A comprehensive literature review on volatile memory forensics," *Electronics*, vol. 13, no. 15, p. 3026, 2024.